```
          guest2 ! Guest.EnterHotel                                    ❷
        }
      }
    }
}
```

❶ Subscribes to the key on creation

❷ Registers to the key when a guest enters the hotel

You can test this by running the following in the root directory of the Git repository:

```
$ sbt "chapter06/testOnly *ReceptionistUsageSpec"
```

The receptionist API allows you to `Find` registered actors as an active retrieval mechanism, not a passive one like the subscription you just saw. `Find` is a one-time message that returns all currently registered actors at that time. You use it in the same way as the subscription: you need an adapter and the service key. Using the same example, you can find the registered actors as follows:

```
   context.system.receptionist ! Receptionist
      .Find(HotelConcierge.goldenKey, listingResponseAdapter)
```

You can deregister an actor by sending a `Deregister` message with the same signature. This way, you can deregister a guest when they leave the hotel:

```
object VIPGuest {

  sealed trait Command
  final case object EnterHotel extends Command
  final case object LeaveHotel extends Command

  def apply() = Behaviors.receive[Command] { (context, message) =>
    message match {
      case EnterHotel => ...
```